

RLSR: A Reinforcement Learning-Based Segment Routing for QoS-Aware Software-Defined Network

WANG WUMIAN¹, SAJAL SAHA² (Senior Member, IEEE), ANWAR HAQUE¹, AND GREG SIDEBOTTOM³

¹Computer Science Department, The University of Western Ontario, London, ON N6A 3K7, Canada

²Computer Science Department, University of Northern British Columbia, Prince George, BC V2N 4Z9, Canada

³Amazon Web Services, Ottawa, ON K1P 5J2, Canada

CORRESPONDING AUTHOR: S. SAHA (sajal.saha@unbc.ca)

ABSTRACT Traffic routing is crucial for the efficient operation of the Internet. As network traffic grows, adaptive and intelligent routing algorithms are needed to meet various Quality of Service (QoS) requirements. Reinforcement Learning (RL)-based routing approaches have demonstrated superior performance over traditional methods. In this study, we propose RL-based Segment Routing (SR), called RLSR, a QoS-aware routing algorithm for Software-Defined Networking (SDN). Our approach ensures loop-free path exploration and leverages learned network QoS states to accelerate convergence when optimizing paths for multiple traffic demands. By integrating SR, RLSR enables flow-based source packet routing while reducing communication overhead between the SDN controller and network switches. Experimental evaluations demonstrate that RLSR improves load balancing by reducing the maximum link utilization from 71.67% to 50% compared with a non-RL baseline. In addition, the proposed knowledge-reuse mechanism accelerates learning, reducing the total number of training episodes by up to 27% compared with a baseline RL configuration without knowledge reuse. Further analysis shows that a higher discount factor ($\gamma = 0.9$) leads to the fastest convergence among the tested settings. These findings highlight the effectiveness of RLSR in optimizing traffic engineering and network performance in dynamic environments.

INDEX TERMS Reinforcement learning (RL), traffic routing, software defined networking (SDN), segment routing (SR), quality of service (QoS).

I. INTRODUCTION

OVER the past decade, global internet connectivity has continued to grow steadily. Recent data indicate that internet users surpassed 5.4 billion in 2024, representing over 67% of the world's population, with projections suggesting continued growth driven by expanding mobile connectivity and digital services in developing regions [1]. Routing—the process of determining and forwarding packets from their source to their destination—is fundamental to end-to-end communication across the internet, which interconnects numerous autonomous networks [2]. Traditionally, traffic routing has relied on protocols such as Open Shortest Path First (OSPF) [3] and the Routing Information Protocol (RIP) [4]. RIP uses a distance-vector mechanism

that considers only hop count when selecting paths [4], which causes it to choose longer but less congested paths over shorter but heavily loaded ones, leading to suboptimal performance under varying traffic conditions. OSPF, one of the most widely deployed interior gateway protocols, computes shortest paths between source–destination pairs based on link weights [5]. To obtain good network performance, link weights in OSPF must be carefully tuned; however, the link weight optimization problem under equal load-sharing has been shown to be NP-hard [6], meaning that finding the globally optimal weight configuration is computationally intractable in general.

To meet increasing performance demands from diverse applications—such as real-time video streaming, cloud

computing, and Internet of Things (IoT) services—new network architectures and traffic-engineering mechanisms have been introduced. Software-Defined Networking (SDN) decouples the control plane from the data plane [7], [8], [9]. In traditional networks, each router maintains its own control and data plane and computes its own forwarding table (e.g., via OSPF) [3], [8]. In contrast, SDN centralizes control logic in a logically centralized controller with a global network view, enabling more flexible interaction with applications, QoS provisioning, and network monitoring [7], [8]. SDN concepts apply not only to wired wide-area networks but also to wireless networks [9]. Segment Routing (SR) [10] is a source-routing architecture in which each packet encodes a list of segments in its header to steer the packet along a specified path [10], [11]. SR's fine-grained control over forwarding supports enhanced network programmability, fast reroute, and load balancing [12].

In parallel, reinforcement learning (RL) has been explored extensively as a means to design adaptive and intelligent routing algorithms [13]. RL-based schemes have been applied to a variety of network types and QoS objectives, often in combination with other techniques such as fuzzy logic [14], [15], game-theory [15], and hierarchical decomposition [13], [16], [17], [18]. Unlike traditional routing methods that rely on static assumptions about network conditions, RL can learn the dynamics of traffic flows and link quality directly from interaction with the environment [18]. This self-learning ability has been used to improve service quality for end users while optimizing network resource utilization [18]. However, many existing RL-based routing approaches do not fully exploit the global view provided by the SDN controller — specifically, they do not leverage the controller's ability to observe network-wide link states and traffic conditions simultaneously to make globally informed routing decisions, nor do they exploit flow-based path enforcement by combining SDN with SR. As a result, these approaches are limited to reactive, locally informed decisions that cannot account for the broader network state, leading to suboptimal load balancing and convergence behavior [18]. In addition, many prior works focus on a single traffic demand at a time and often require predefined paths or initial routing solutions (e.g., from OSPF) as input [18].

The objective of this work is to develop an RL-based routing algorithm for SDN that addresses these limitations. Our design goals are to develop a lightweight and efficient algorithm that (i) allows users to express customized QoS preferences for path selection and (ii) minimizes network costs during the learning process, while still producing routing paths that align with user-defined QoS requirements. The main contributions of this paper are summarized as follows:

- We modify State–Action–Reward–State–Action (SARSA) [19], specifically the standard single-step on-policy variant, to perform *aggregate action selection* at the episode level. By first constructing the entire exploratory path and then executing it, the proposed scheme reduces the number of message exchanges

(e.g., probe packets and QoS feedback) between the SDN controller and network switches compared with the standard per-step SARSA interaction pattern.

- During learning, the proposed algorithm ensures that packets do not become trapped in forwarding loops. Each packet either reaches its destination or stops being forwarded once all candidate next-hop nodes on its current path have been visited. Consequently, a packet is forwarded to any node in the network at most once per episode.
- We introduce a dual-reward mechanism that decomposes each action's reward into *local* and *global* components. Local rewards are used to identify high-quality paths for individual traffic demands, whereas global rewards reflect network-wide conditions such as link utilization. The global rewards are accumulated in a global Q-table that can be used to initialize local Q-tables for new demands, thereby accelerating convergence.
- The proposed RLSR-Routing algorithm does not require prior knowledge of the network, predefined paths, or initial link weights. It can autonomously learn network conditions and sequentially assign a list of traffic demands to routes across the network.

We evaluate RLSR-Routing against a routing algorithm currently used in production by a major telecom solutions provider. Due to confidentiality constraints, the provider cannot be explicitly named and further implementation details of their algorithm cannot be disclosed. Experiments are conducted on a mesh topology with randomly generated traffic demands. We compare the quality of the selected paths in terms of maximum link utilization. The results show that RLSR-Routing reduces maximum link utilization and, when handling a sequence of traffic demands, achieves faster convergence by reusing knowledge acquired from previous learning episodes.

The remainder of this paper is organized as follows. Section II defines the problem statement and network model. Section III reviews related work on RL-based routing across non-SDN, SDN, and deep RL approaches. Section IV presents a high-level overview of the proposed RLSR-Routing methodology. Section V details the RL formulation, including the state and action space and reward mechanism. Section VI presents the proposed RLSR-Routing algorithm, including aggregate action selection and the local and global reward design. Section VII describes the end-to-end workflow of RLSR-Routing. Section VIII reports the experimental results, including a comparative study with a non-RL routing algorithm and a convergence analysis. Section IX concludes the paper and outlines directions for future work.

II. PROBLEM STATEMENT

In this work, we focused on traffic routing in SDN; therefore, we define our network as a set of SDN switches with an SDN controller that has a global view of the network. This allows the controller to know the topology of the network,

including every node and link. The controller has dedicated links to communicate with network nodes, but neither the controller nor these links are used for routing packets between the network's end users. Our routing algorithm interacts with the SDN controller to perform path calculation and installs defined routing paths into the network.

From the routing algorithm's point of view, a network can be abstracted as a directed graph, $G(V, E)$, where V is the set of nodes that each corresponds to a physical device (SDN switch), and E is the set of links that connects nodes. $l_{i,j}$ denotes the unidirectional link from node i to j where $i, j \in V$. We define a traffic demand as a flow of data transmitted from a source node to a destination node. One source and destination pair can have multiple traffic demands, each with a specified amount of data as demand traffic (expressed in bits/second). However, each traffic demand can only be assigned one routing path, which is defined as a list of nodes that connects from source to destination.

Each link and node of a network has a maximum capacity. For a link, we use the term "maximum bandwidth" (in bits/second) to represent the maximum traffic that can travel through the link, and "used bandwidth" (in bits/second) to represent the current amount of data on the link. A node's processing rate (in bits/second) is the maximum amount of incoming data it can handle. In addition, all links have a field called "link reliability", which varies from 0% to 100%. The higher the link reliability, the lower the chances of link failure will occur.

Based on the above settings, our research objective is to give a network with SDN architecture with all nodes supporting Segment Routing (SR) and develop an RL-based routing algorithm that can find user-preferred paths for traffic demands. Our definition of 'user-preferred' paths are those paths found by our routing algorithm based on weights of one or more Quality of Service (QoS) parameters, which are adjusted by our algorithm's users. The following assumptions are made:

- Communications between SDN controllers and network switches are through dedicated links, in a reliable transfer manner.
- SDN controllers are not involved in routing packets that originated from the network's end users. To improve visual clarity, the SDN controller and its dedicated links to network nodes are omitted from topology diagrams that focus on routing path illustration, as their inclusion would obscure the routing paths under discussion. In contrast, figures that illustrate the RLSR-Routing architecture and component interactions explicitly show the controller and its connections, as these are central to what those figures convey.
- All SDN switches of the networks considered in this research support SR, i.e., they can handle a packet's SR header and execute segments properly.
- All nodes are reliable, and if total incoming traffic does not exceed a node's processing rate, no packets will be dropped by nodes.

- For each pair of adjacent nodes $i-j$, at most two links exist: link $i \rightarrow j$ and link $j \rightarrow i$, directly connecting i and j .
- If a link's used bandwidth does not exceed its maximum bandwidth and link failure does not occur, no packet will be lost when traveling on that link.

III. RELATED WORK

Recent advancements in RL have led to its adoption across various network routing paradigms, including traditional non-SDN networks, SDN-based architectures, and emerging DRL-based approaches. This section provides a critical review of existing RL-based routing techniques, explicitly identifying their strengths, limitations, and the research gaps that motivate our proposed approach. Crucially, among the reviewed SDN-based works, QAR [16] is the most closely related to our approach: RLSR-Routing directly extends QAR by addressing two fundamental limitations — its per-step communication overhead and its inability to reuse learned network knowledge across traffic demands. The discussion is organized into three categories: RL for non-SDN networks (Subsection III-A), RL for SDN (Subsection III-B), and DRL-based routing (Subsection III-C), followed by a synthesis of identified gaps (Subsection III-D).

A. RL ON NON-SDN NETWORKS

Several recent studies have focused on RL approaches for routing in Wireless Sensor Networks (WSNs) [20], [21], [22]. A key characteristic of WSNs is that network devices are battery-powered, making energy conservation a primary optimization objective. Both [20] and [21] exchange energy-related information by embedding extra fields in data packets — in [20], the sender node's current Q-value and the previous node's energy level are forwarded to the next-hop node, while [21] designs energy-aware QoS routing using RL with a focus on network lifetime. Although effective in resource-constrained environments, these approaches incur additional communication overhead from the exchange of energy metadata, which can itself accelerate energy depletion — a fundamental tension in WSN design. In [22], Q-learning is combined with transmission gradient optimization to balance energy consumption; each node maintains an estimated hop count to the base station and shares Q-values via status packets. While [22] reduces the per-packet overhead relative to [20] and [21], all three approaches are constrained to distributed, non-centralized architectures that lack a global network view — a limitation that SDN-based approaches directly address. None of these works consider flow-based path control or reuse of learned knowledge across demands, which are core contributions of our proposed RLSR-Routing.

Ad-hoc Networks (ANETs) lack centralized controllers, feature wireless connections, and exhibit dynamic topologies. Mobile Ad-hoc Networks (MANETs) are a subtype where nodes exhibit mobility; common types include Vehicular Ad-hoc Networks (VANETs) for vehicles operating

in areas such as cities [18]. All reviewed ANET papers use packet loss, retransmission rate, or delivery ratio as a primary performance metric, reflecting the inherent unreliability of wireless links. Unlike earlier RL-only approaches, many combine RL with other techniques to compensate for the absence of a central controller. QLMAODV [23] integrates Q-learning with AODV [24] — a loop-free MANET routing protocol — to improve path stability. Its strength lies in inheriting AODV’s loop-freedom guarantee; however, it requires periodic route discovery broadcasts, generating significant control overhead in dense networks. PFQ-AODV [14] extends this by incorporating fuzzy logic for bandwidth, mobility, and link quality estimation, and improves vehicle movement prediction using neighbor information rather than position alone. While more adaptive, its complexity increases with network density. QGeo [25] accounts for mobility through periodic location exchange via HELLO packets, but this introduces regular broadcast overhead that scales poorly in large or high-mobility networks. Critically, none of these approaches exploit a centralized network view or support flow-based routing — both of which are enabled by SDN and leveraged in RLSR-Routing to reduce overhead and improve path quality.

Authors in [15] propose combining fuzzy logic, game theory, and RL: fuzzy logic clusters vehicles and selects cluster heads, game theory coordinates inter-cluster communications, and RL evaluates routing paths within clusters. While the hierarchical structure reduces individual node overhead, it introduces cluster formation complexity and single points of failure at cluster heads. Q2-R [26] bootstraps initial paths before Q-learning, allowing the agent to start from non-trivial solutions and gradually refine Q-values — an approach that partially motivated our global Q-table design, which similarly provides prior knowledge to accelerate convergence. However, unlike Q2-R, our global Q-table accumulates network-wide QoS knowledge reusable across different traffic demands, rather than bootstrapping from a fixed initial path. The hierarchical grid scheme in [27] offers scalable geographic routing without cluster heads, but like the others, it lacks centralized coordination and flow-level path control.

Authors in [28] and [29] focus on Wireless Mesh Networks (WMNs), where multi-interface nodes act as gateways. In [28], Q-learning selects among predefined routing algorithms rather than computing paths directly — a design that limits adaptability to scenarios not covered by the predefined set. In [29], gateway selection and next-hop decisions are learned separately, reducing coordination but increasing the risk of suboptimal end-to-end paths. RLOR [30] targets multi-hop wireless networks for video streaming, combining RL with opportunistic routing to make next-hop decisions on the fly. While effective for broadcast-natured wireless links, opportunistic routing is unsuitable for SDN environments where deterministic, flow-based path control is preferred. Collectively, these non-SDN approaches demonstrate RL’s flexibility but consistently lack the global

network view, centralized control, and flow-based routing that SDN provides and that RLSR-Routing exploits.

B. RL ON SDN

SDN’s logically centralized controller provides a global network view that fundamentally changes what RL agents can observe and control compared to distributed non-SDN architectures [31]. Among SDN-based RL routing algorithms, QAR [16] is the most directly relevant to our work and serves as the primary baseline for RLSR-Routing. QAR proposes a QoS-aware hierarchical SDN routing algorithm using SARSA — an on-policy RL algorithm where Q-value updates depend on the action actually performed at the next state, rather than the maximum possible action as in Q-learning. This makes SARSA more suitable for online learning in routing scenarios where the agent must follow its own policy during training. We extend QAR rather than other reviewed works for two specific reasons: (i) QAR’s SARSA-based formulation is directly compatible with our aggregate action selection modification, and (ii) QAR’s Q-table structure is naturally extensible to our dual local/global reward design. However, QAR has two key limitations that RLSR-Routing addresses: it requires one control-plane round-trip per action step, generating $2n$ communications for an n -hop path; and it reinitializes its Q-table from scratch for every new traffic demand, discarding previously learned network knowledge. RLSR-Routing reduces communications to $n + 1$ per episode through aggregate action selection, and preserves learned knowledge via a persistent global Q-table.

VS-routing [32] addresses SARSA’s fixed exploration rate by applying a variable ϵ -greedy policy that adapts with hop count, reducing unnecessary exploration in later stages. While this improves consistency, VS-routing uses no immediate rewards, which slows early convergence. Work in [33] targets controller load-balancing rather than path routing, using Bayesian-enhanced ϵ -greedy to bias exploration toward frequently selected actions. For Q-learning-based SDN routing, [34] updates all edge rewards per episode based on bandwidth utilization rather than selected actions — reducing per-step overhead but sacrificing action-level granularity. Reference [35] applies Q-learning to wireless SDN load-balancing, while QR-SDN [36] selects paths from a precomputed set rather than computing them directly — a design that guarantees path validity but limits adaptability to paths not in the predefined set. Works in [37] and [38] apply Q-learning to distributed SDN variants, optimizing link weights [37] or protocol selection [38] rather than routing paths directly. None of these works support knowledge reuse across demands or reduce control-plane overhead through aggregate action selection.

C. DEEP RL-BASED ROUTING

Using RL for traffic routing dates back to the 1990s [39]; DRL-based approaches are comparatively recent, emerging primarily after 2017. Although DRL approaches such

TABLE 1. Comparison of RLSR-routing with closely related baselines.

Feature	RLSR (Ours)	QAR [16]	ENERO [42]	WA-SRTE [41]
RL Algorithm	SARSA (tabular)	SARSA (tabular)	DRL + GNN (DQN)	DRL (offline)
Path Construction	From scratch	From scratch	Predefined k -shortest paths	OSPF-initialized SR paths
Action Selection	Aggregate (episode-level)	Per-step	Per-demand (offline DRL)	Offline phase + LP (online)
Control-Plane Overhead	$n + 1$ per n -hop path	$2n$ per n -hop path	N/A (offline training)	N/A (offline training)
Loop-Free Guarantee	Yes (during training)	No	Yes (predefined paths)	Yes (predefined paths)
SDN Integration	Yes	Yes (hierarchical SDN)	Yes (SDN-based WAN)	No (IP/SRv6 network)
Segment Routing	Yes (SR for forwarding)	No	Yes (SR intermediate node)	Yes (SRv6)
Knowledge Reuse Across Demands	Yes (global Q-table)	No	No	No
Offline Training Required	No	No	Yes	Yes (offline design phase)
Prior Path/Weight Knowledge Required	No	No	Yes (initial OSPF weights)	Yes (historical TM + OSPF)
External ML Library Required	No	No	Yes (GNN + TensorFlow)	Yes (DRL framework)

as DROM [40], WA-SRTE [41], and ENERO [42] have demonstrated strong performance, they share a common limitation relevant to our work: they require external neural network libraries and significant training data or offline precomputation, making deployment in lightweight or resource-constrained SDN environments challenging. ENERO [42], for instance, requires initial OSPF weights and a predefined traffic demand set as input, while WA-SRTE [41] fixes link weights and SR-node placement after an offline training phase, limiting adaptability to dynamic traffic changes. In contrast, RLSR-Routing uses a tabular SARSA-based formulation that requires no external ML libraries, no offline training phase, and no prior knowledge of link weights — making it directly deployable on an SDN controller with minimal infrastructure requirements. We therefore build upon QAR’s RL formulation rather than adopting a DRL approach, prioritizing deployability and interpretability over potential performance gains that DRL may offer at scale. DROM [40] adjusts link weights dynamically to reroute traffic, while WA-SRTE [41] combines offline DRL with online linear programming across two separate phases. Deep Q-learning in [43] classifies flows into mice and elephant types, training separate networks for each — effective but requiring significant architectural complexity. Reference [17] proposes hierarchical deep double Q-routing with recursive cluster-based path construction, adapting the double Q-learning technique [44] to reduce overestimation bias.

D. ANALYSIS AND CURRENT RESEARCH GAP

Most reviewed papers propose RL-based routing algorithms, though DRL approaches have attracted growing interest over the past five years. Among non-SDN studies, the majority target wireless and/or ad-hoc networks [14], [15], [20], [21], [22], [23], [25], [26], [27], [28], [29], [30]. Non-SDN RL routing generates additional communication overhead from inter-node status exchanges — for example, [20], [21], and [22] require nodes to broadcast energy levels and Q-values, consuming the same battery resources they aim to conserve. SDN controllers’ global network visibility eliminates this distributed exchange overhead entirely, as demonstrated by QAR [16] and related SDN-based works [32], [33], [34].

A significant portion of reviewed papers propose composite protocols integrating RL or DRL with other techniques. The primary motivations are: accelerating convergence (e.g., QAR [16] uses hierarchical decomposition; Q2-R [26] uses bootstrapping); ensuring loop-free routing (e.g., QLMAODV [23] inherits AODV’s loop-freedom guarantee [24]); and enabling source routing (e.g., ENERO [42], WA-SRTE [41]). Notably, QAR [16] does not guarantee loop-free routing during the learning phase despite using an SDN controller as the RL agent — RLSR-Routing addresses this directly through aggregate action selection, which constructs only acyclic paths by tracking visited nodes within each episode.

No reviewed SDN-based RL work computes paths for multiple parallel traffic flows or reuses learned network knowledge across sequential demands. These two gaps directly motivate RLSR-Routing: aggregate action selection reduces per-episode control-plane communications from $2n$ to $n+1$ for an n -hop path compared to standard SARSA [16], and the global Q-table preserves learned QoS knowledge across demands, reducing total convergence episodes by up to 27% in our experiments. Table 1 summarizes these distinctions concisely across the five most closely related works.

IV. HIGH LEVEL OVERVIEW OF PROPOSED METHODOLOGY

As the number of Internet users and types of services that rely on networking increases, traditional distance vector or shortest path based algorithms, such as RIP [4] and OSPF [3], fail to provide optimal network performance under dynamic traffic conditions and diverse QoS requirements [6]. Previous studies demonstrate the potential of using RL or DRL to develop intelligent routing algorithms that are adaptive to dynamic network topology and fulfill a variety of QoS requirements [13], [18]. To the best of our knowledge, existing approaches do not fully exploit SDN architecture combined with a flow-based source packet routing paradigm [18].

In this research, we propose RLSR-Routing, an RL-based routing algorithm over SR-enabled SDN architecture as shown in Fig. 1a to address the current research gap.

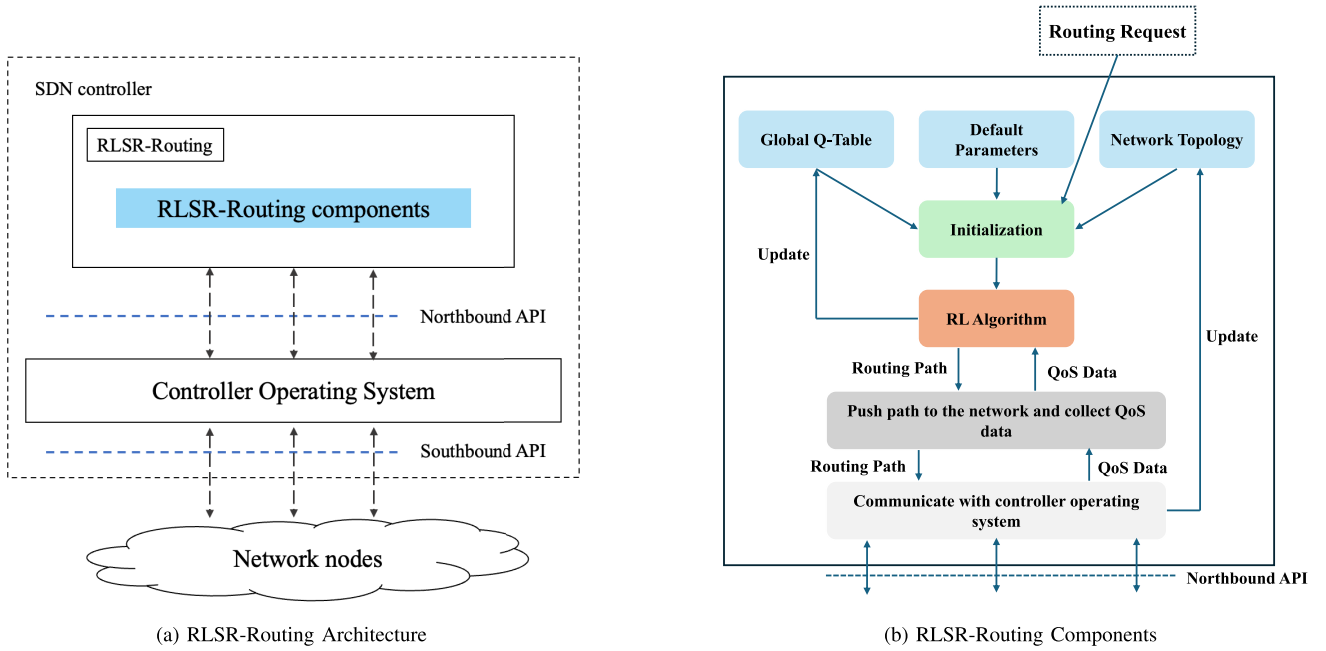


Fig. 1. Overview of the RLSR-Routing framework.

RLSR-Routing reduces network operation costs during the path-finding process by aggregating action selection within each episode, eliminating the per-step round-trip communication overhead inherent in standard SARSA-based approaches [16]. Furthermore, by reusing previously accumulated network knowledge via a global Q-table, RLSR-Routing achieves faster convergence than QAR [16], a SARSA-based routing algorithm that reinitializes its Q-table from scratch for every new traffic demand. This section describes the RLSR-Routing framework and the rationale behind its design.

Although different literature may categorize routing as an application plane or control plane function [2], [7], such a difference does not affect RLSR-Routing’s framework, because in either case RLSR-Routing does not directly interact with network devices. Fig. 1b illustrates RLSR-Routing’s internal components and their interactions.

As shown in Fig. 1b, a routing request enters RLSR-Routing from the top and triggers the initialization phase, in which the RL algorithm draws on three storage components: the *Global Q-table*, which provides previously learned network QoS knowledge; *Default parameters*, which supply default QoS weights and hyperparameters; and *Network topology*, which provides the current graph representation of the network. Once initialized, the RL algorithm iteratively constructs candidate paths and passes them downward to the communication layer. The “Push path to the network and collect QoS data” block forwards each candidate path to the “Communicate with controller operating system” block, which installs the path in the network via the Northbound API and retrieves per-hop QoS measurements in return. These measurements flow back upward to the RL algorithm,

which uses them to compute rewards and update both the local and global Q-tables (upward arrows labeled “Update”). Once the algorithm converges, the final routing path is delivered to the SDN controller for installation.

Based on their roles in this workflow, RLSR-Routing’s components fall into three categories: (1) **Storage** — the Global Q-table, Default parameters, and Network topology; (2) **RL algorithm** — the central path-finding and learning component; and (3) **Communication** — the two blocks that interface with the SDN controller to execute paths and collect QoS feedback.

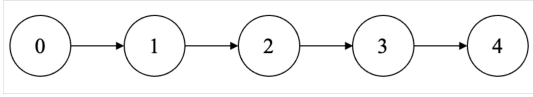
V. RL FORMULATION FOR OUR PROPOSED RLSR-ROUTING

The RL algorithm used in RLSR-Routing can be viewed as an extension to previous work, particularly QAR from [16]. RLSR-Routing applies a modified SARSA, an on-policy RL algorithm in which the agent learns by interacting with the environment through a complete Markov Decision Process (MDP). Formally, the MDP is defined as a tuple $\langle S, A, P, R \rangle$, where S is the set of network states (one per node), A is the set of forwarding actions (one per directed link), P is the state transition function determined by the network topology, and R is the QoS-based reward function.

In this Section, we define the state space, action space, and reward mechanism for RLSR-Routing’s RL algorithm. We also introduce two modifications made to standard SARSA that are central to our approach: (i) *aggregate action selection*, which selects the entire action sequence for an episode before execution, reducing the number of control-plane interactions compared with the standard step-by-step SARSA workflow; and (ii) a *dual-reward scheme*,

TABLE 2. Summary of QoS-based reward components in RLSR-routing.

Reward Component	QoS Category	Formula
Hop Count Reward (R_{hop})	Latency	$R_{\text{hop}} = \frac{1}{\text{number of hops}}$
Transmission Rate Reward ($R_{\text{transmission}}$)	Latency	$R_{\text{transmission}} = \frac{2}{\pi} \arctan(\text{node } i \text{ processing rate})$
Link Reliability Reward ($R_{\text{reliability}}$)	Reliability	$R_{\text{reliability}} = \text{link } l \text{'s reliability value}$
Traffic Intensity (Current) (R_{inten})	Load Balancing	$R_{\text{inten}} = 1 - \frac{\text{Node } j \text{ current incoming traffic}}{\text{Node } j \text{ processing rate}}$
Traffic Intensity (Estimated) ($R_{\text{inten-est}}$)	Load Balancing	$R_{\text{inten-est}} = 1 - \frac{\text{Node } j \text{ estimated incoming traffic}}{\text{Node } j \text{ processing rate}}$
Link Utilization (Current) (R_{util})	Load Balancing	$R_{\text{util}} = 1 - \frac{\text{Link } l \text{ current used bandwidth}}{\text{Link } l \text{ maximum bandwidth}}$
Link Utilization (Estimated) ($R_{\text{util-est}}$)	Load Balancing	$R_{\text{util-est}} = 1 - \frac{\text{Link } l \text{ estimated used bandwidth}}{\text{Link } l \text{ maximum bandwidth}}$

**Fig. 2. Trivial topology 1 (T1).**

which computes both a local reward to guide per-demand path selection and a global reward to accumulate reusable network-wide QoS knowledge across demands. The design of these modifications are presented in detail in Section VI.

A. STATE AND ACTION SPACE

RLSR-Routing operates on a per-demand basis, where each episode corresponds to forwarding a single packet from the source to the destination defined by the traffic demand.

Let S denote the state space, which is the set of all possible network states. Since each state corresponds to a network node currently holding the packet, we have $|S| = |V|$, where V is the set of network nodes. A state $s \in S$ indicates that node s is holding the packet. We also use the notations s_t and s_{t+1} to represent the state at time t and time $t + 1$, respectively.

Similarly, let A denote the action space, which is the set of all possible forwarding actions. Since each action corresponds to transmitting the packet over a directed link, we have $|A| = |E|$, where E is the set of directed links in the network. An action $a_{i,j} \in A$ represents forwarding the packet along link (i, j) , i.e., from node i to node j . The notations a_t and a_{t+1} denote the actions selected at time t and $t + 1$, respectively.

To illustrate, Fig. 2 shows a trivial topology T_1 consisting of five nodes and four unidirectional links. The SDN controller and its connections are omitted for simplicity. Suppose a packet is transmitted from node 0 to node 4. At time t_0 , the packet resides at node 0, which corresponds to state s_0 , and the selected action is to forward the packet along link $l_{0,1}$. At the next step, time t_1 , the packet reaches node 1, yielding state s_1 . This process continues until the destination node is reached.

B. REWARD MECHANISM

The reward components used in RLSR-Routing are summarized in Table 2. Five QoS parameters are considered,

grouped into three categories: latency, reliability, and load balancing. Assuming the performed action forwards one packet from node i to node j through link l , each reward component is defined as follows.

1) *Latency-Related Rewards*: Two metrics capture latency. The hop-count reward R_{hop} assigns a higher reward to shorter paths, since fewer hops generally correspond to lower end-to-end delay. The transmission-rate reward $R_{\text{transmission}}$ reflects the processing rate of node j : a higher processing rate reduces the time required to transmit all bits of a packet onto the link, and therefore yields a higher reward. These two metrics are kept separate because they capture distinct aspects of latency: R_{hop} reflects path-level delay, while $R_{\text{transmission}}$ reflects node-level delay.

2) *Reliability-Related Reward*: The reliability reward $R_{\text{reliability}}$ is based on the probability that link l remains operational. Since all network switches and SDN controllers are assumed to be fully reliable, only the possibility of link failure is considered.

3) *Load-Balancing-Related Rewards*: Two metrics capture load balancing: traffic intensity (R_{inten} , $R_{\text{inten-est}}$) and link utilization (R_{util} , $R_{\text{util-est}}$). For both metrics, current and estimated values are considered — the estimated values reflect projected conditions after the traffic demand's flow is placed on the link. Actions that route traffic over less-congested links receive higher rewards.

All reward components are normalized to $[0, 1]$, where a value closer to 1 indicates better QoS for the corresponding metric. To illustrate how these components are computed in practice, consider topology T_1 , in which a packet is forwarded from node 0 to node 4 along the path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$. All links have a maximum bandwidth of 10 Mb/s and a reliability of 95%, while all nodes operate at a processing rate of 50 Mb/s. The link from node 3 to node 4 currently carries 5 Mb/s, and the new demand contributes an estimated 0.5 Mb/s.

Table 3 reports the reward values computed for the action of forwarding from node 3 to node 4. Each value reflects a single QoS dimension: for example, $R_{\text{util}} = 0.5$ indicates that the link is currently at 50% capacity, while $R_{\text{reliability}} = 0.95$ reflects that the link fails 5% of the time. Together, these values illustrate how the RL agent evaluates a routing action from a multi-dimensional QoS

TABLE 3. Example reward values for the action from node 3 to node 4.

Reward Component	Value	Explanation
Hop Count Reward (R_{hop})	0.25	Link $l_{3,4}$ is the 4th hop in the path.
Transmission Rate Reward ($R_{\text{transmission}}$)	0.9873	Computed using node 3's processing rate: $\frac{2}{\pi} \arctan(50)$.
Link Reliability Reward ($R_{\text{reliability}}$)	0.95	Link reliability is 95%.
Traffic Intensity (Current) (R_{inten})	0.9	Current incoming traffic at node 4 is 5 Mb/s.
Traffic Intensity (Estimated) ($R_{\text{inten-est}}$)	0.89	Expected incoming traffic becomes 5.5 Mb/s.
Link Utilization (Current) (R_{util})	0.5	Current bandwidth usage is 5 Mb/s.
Link Utilization (Estimated) ($R_{\text{util-est}}$)	0.45	Expected usage after demand becomes 5.5 Mb/s.

perspective — a high-scoring action will have values close to 1 across all components.

VI. PROPOSED RLSR-ROUTING

This section presents the two key algorithmic modifications introduced in RLSR-Routing: aggregate action selection and a dual-reward design. Subsection VI-A describes how aggregating action selection across an entire episode reduces control-plane overhead while preserving the correctness of the SARSA update rule. Subsection VI-B introduces the dual-reward scheme and the associated local and global Q-tables, which together enable efficient per-demand path optimization and long-term accumulation of network knowledge.

A. AGGREGATE ACTION SELECTION

This subsection presents one of the key modifications introduced in RLSR-Routing: aggregate action selection within each episode. Prior studies indicate that Q-learning and SARSA are the predominant reinforcement learning (RL) approaches for traffic-engineering problems. Both Q-learning and SARSA follow a similar five-step interaction pattern, differing only in the logic used for Step 4: Q-learning updates $Q(s_t, a_t)$ using the *maximum* Q-value over all possible next actions (off-policy), whereas SARSA updates $Q(s_t, a_t)$ using the Q-value of the *action actually selected* at the next state (on-policy):

- 1) At time t , select an action a_t from state s_t using an exploration policy.
- 2) Execute action a_t in the environment.
- 3) Observe the reward R and the next state s_{t+1} .
- 4) Update the Q-value $Q(s_t, a_t)$.
- 5) Set $t \leftarrow t + 1$ and continue.

This stepwise process resembles a “stop-and-wait” mechanism: the agent cannot choose the next action until the current action is executed and its reward becomes available. In contrast, RLSR-Routing performs *aggregate action selection*, where the entire action sequence for an episode is selected before any action is executed. The modified workflow is:

- 1) From the initial state s_0 , select an action $a_{0,1}$ leading to an unvisited next state s_1 .
- 2) Append $a_{0,1}$ to the action list.

- 3) Repeat the selection process from $s_1, s_2, \dots$, until the destination is reached or no unexplored transitions remain (avoiding loops).
- 4) Execute the ordered action list $\{a_{0,1}, a_{1,2}, a_{2,3}, \dots\}$ in the network.
- 5) Collect QoS measurements and compute rewards in the same order.
- 6) Update the Q-values for all executed state–action pairs sequentially.

Thus, the agent determines the full exploration path upfront and then performs the actions in batch, reducing control-plane interactions and improving learning efficiency. This design is justified by the following observations.

Observation I: Updating $Q(s_t, a_t)$ does not affect the selection of a_{t+1} if a_{t+1} leads to a previously unseen state within the same episode.

Observation II: Updating $Q(s_{t+1}, a_{t+1})$ does not modify the correctness of previously updated Q-values provided that (i) updates follow the order of action execution, and (ii) the constructed episode path is non-cyclic. Under the SARSA update rule,

$$Q_{t+1}(s_t, a_t) = (1 - \alpha)Q_t(s_t, a_t) + \alpha(R + \gamma Q_t(s_{t+1}, a_{t+1})), \quad (1)$$

where $\alpha \in (0, 1]$ is the learning rate controlling how much new information overrides old estimates, $\gamma \in [0, 1]$ is the discount factor determining the importance of future rewards, R is the immediate reward received after executing action a_t in state s_t , and s_{t+1}, a_{t+1} , each update depends only on the most recent estimate of $Q_t(s_t, a_t)$ and the Q-value for the next state–action pair. Because aggregate action selection produces an acyclic sequence of states, each state appears at most once per episode, and sequential Q-value updates remain independent and consistent. This enables RLSR-Routing to decouple action selection from reward computation while maintaining the theoretical correctness of the SARSA update rule.

B. LOCAL AND GLOBAL REWARD DESIGN

Beyond aggregate action selection, RLSR-Routing also enhances learning efficiency by introducing a dual-reward mechanism. Existing RL-based routing schemes, such as QAR [16], typically follow the workflow:

Algorithm 1 Find Route Using Reinforcement Learning

```

1: function RL_FINDROUTE(TrafficDemand  $D$ , Boolean  $B$ , QoSWeights  $W$ , Hyperparameters  $H$ )
2:   // Initialization
3:   Obtain graph  $G$  from RLSR-Routing
4:   Obtain global Q-table  $GT$  from RLSR-Routing
5:   Obtain default QoS weights  $W_d$  and hyperparameters  $H_d$ 
6:   QTable  $localT \leftarrow INITLTABLE(G, GT, B)$ 
7:   QoSWeights  $w_l \leftarrow (W = \text{NULL})?W_d : W$ 
8:   Hyperparameters  $h_l \leftarrow (H = \text{NULL})?H_d : H$ 
9:    $episodes \leftarrow 0$ 
10:  while  $episodes < h_l.E$  do
11:    Path  $tempPath \leftarrow \text{FINDTEMPPATH}(D, localT, h_l$ 
12:    PUSH_PATH_FOR_ROUTING( $tempPath$ )
13:    QoSData[]  $qData$ ; int  $count \leftarrow \text{COLLECT\_QoS\_DATA}(tempPath)$ 
14:    Reward[]  $lRewards \leftarrow \text{CALCULATELREWARDS}(qData, w_l, count, D)$ 
15:    Reward[]  $gRewards \leftarrow \text{CALCULATEGREWARDS}(qData, W_d, count$ 
16:    UPDATETABLE( $localT, lRewards, h_l, count$ )
17:    UPDATETABLE( $GT, gRewards, H_d, count$ )
18:     $episodes \leftarrow episodes + 1$ 
19:  end while
20:  return FINDFINALPATH( $D, localT, h_l$ )
21: end function

```

- 1) Extract the source–destination pair from the traffic demand.
- 2) Initialize a Q-table (often with all zero entries).
- 3) Execute multiple episodes of the Markov decision process.
- 4) After convergence, derive the final routing path from the learned Q-table.

Although effective, this approach discards previously learned information when a new traffic demand arrives, as the Q-table is fully reinitialized. Consequently, useful knowledge about underlying network QoS conditions is lost, requiring the RL agent to relearn common network characteristics from scratch. Motivated by approaches that partially retain prior learning [26], RLSR-Routing introduces a *dual-reward scheme* in which each executed action yields both a *local reward* and a *global reward*. The local reward guides the search for a high-quality path for the current traffic demand, reflecting user-defined QoS preferences. The global reward captures long-term network characteristics (e.g., reliability, intensity, utilization) that are beneficial across different demands.

This dual-reward structure results in two separate Q-tables:

- **Local Q-table:** Used for action selection within the current episode. It is updated using local rewards and reflects path preference for the specific traffic demand.
- **Global Q-table:** Serves as persistent memory recording learned network conditions across episodes and across traffic demands.

During initialization, users may choose to seed the local Q-table with the global Q-table rather than starting from zeros. This accelerates convergence by providing

prior knowledge of link quality. During learning, only the local Q-table influences action selection, while the global Q-table is updated silently in the background. This separation also facilitates potential multi-threaded extensions of RLSR-Routing, where multiple RL agents may operate concurrently, as the persistent global Q-table can be treated as an independent shared component (see Fig. 1b).

VII. PROPOSED RLSR-ROUTING WORKFLOW

This section presents the end-to-end workflow of the proposed RLSR-Routing algorithm. Given a traffic demand and optional user-defined parameters, the algorithm iteratively explores candidate paths, executes them in the SDN network, collects QoS feedback, and updates both local and global Q-tables. After a fixed number of training episodes, a deterministic routing path is derived from the converged local Q-table. The overall procedure is summarized in Algorithm 1.

Algorithm 1 captures the complete learning pipeline: initialization of all data structures (Lines 3–8), temporary-path construction via FindTempPath (Line 11), path execution and QoS observation via Push_Path_for_Routing and Collect_QoS_Data (Lines 12–13), local and global reward computation via CalculateLRewards and CalculateGRewards (Lines 14–15), Q-table updates via UpdateTable (Lines 16–17), and final path extraction via FindFinalPath (Line 19).

A. INITIALIZATION PHASE

Before learning starts, the RL agent initializes the local Q-table, QoS weights, hyperparameters, and the traffic demand to be served. The local Q-table can either reuse

TABLE 4. QoS weights and constants used in the algorithm.

Variable	Description
l_{Constant}	Constant used when calculating local reward
g_{Constant}	Constant used when calculating global reward
W_c	Hop-count reward weight
W_t	Transmission-rate reward weight
W_r	Link-reliability reward weight
W_i	Traffic-intensity reward weight
W_u	Link-utilization reward weight

TABLE 5. Hyperparameters used in algorithm 1.

Parameter	Value	Description
ε	0	Exploration rate used in the ε -greedy action selection strategy; set to 0 to disable exploration.
α	0.9	Learning rate that controls how quickly the model updates its knowledge using new information.
γ	0.9	Discount factor that determines the importance of future rewards in the reinforcement learning process.
TTL	32	Time-To-Live, representing the maximum number of hops or steps allowed per episode.
E	75	Total number of training episodes used during learning.

previously learned knowledge stored in the global Q-table or be initialized from scratch.

Both global and local Q-tables are implemented as two-dimensional arrays, where entry $[i][j]$ represents the Q-value for the state-action pair “at node i , forward to node j ”. When global reuse is enabled ($B = \text{true}$), the local Q-table is initialized as a deep copy of the global Q-table; otherwise, non-existent links are assigned a minimal value, and existing links start from zero. QoS weights and hyperparameters are either taken from user input or default values, as summarized in Tables 4 and 5.

Users can tune the QoS weights to emphasize different performance objectives (see Table 4 for the full list of weights and constants). The constants l_{Constant} and g_{Constant} are derived from these weights according to the local and global reward formulations defined in Equations (4) and (5).

$$l_{\text{Constant}} = W_c + W_t + W_r + W_i + W_u + 0.1 \quad (2)$$

$$g_{\text{Constant}} = W_r + W_i + W_u. \quad (3)$$

The additive 0.1 term in l_{Constant} ensures that a reward of exactly zero is never achievable even when all individual QoS reward components attain their maximum value of 1. This design choice prevents Q-values from stagnating at zero during early training episodes, maintaining a non-trivial gradient for Q-value updates throughout the learning process.

B. TEMPORARY PATH CONSTRUCTION

Instead of selecting and applying a single action at each step (as in standard SARSA), RLSR-Routing first constructs a non-cyclic exploratory path for the entire episode. This aggregate action selection reduces control-plane overhead and ensures that packets do not revisit the same node within one episode.

The function FIND TEMP PATH starts from the source node of the traffic demand and repeatedly selects an unvisited neighbor using an ε -greedy policy, until the destination is reached, a dead end is encountered, or the TTL (Time to Live) constraint is met. The resulting path is guaranteed to be acyclic within the episode.

C. PATH EXECUTION AND QOS OBSERVATION

Once a temporary path is constructed, it is encoded into the packet header and installed via the SDN controller. The controller forwards the packet along the path through SR-enabled switches and collects per-hop QoS data, such as reliability, traffic intensity, and link utilization.

In the current study, this process is simulated; in a real deployment, the SR header may include an additional flag (e.g., “SEND QoS DATA”) to distinguish learning packets. The controller then returns an ordered QoS trace for the executed path. If packet loss is detected using a timeout mechanism, the last action is marked as unsuccessful, and the action sequence is truncated accordingly.

D. REWARD COMPUTATION

For each executed path, RLSR-Routing computes both local and global rewards. Local rewards capture path quality for the current traffic demand, while global rewards encode long-term network conditions that can be reused across demands. The individual reward components and their formulas are defined in Table 2, and the QoS weights are listed in Table 4. The local and global rewards are computed as:

$$R_{\text{local}} = W_c R_{\text{hop}} + W_t R_{\text{transmission}} + W_r R_{\text{reliability}} + W_i R_{\text{inten-est}} + W_u R_{\text{util-est}} - l_{\text{Constant}} \quad (4)$$

$$R_{\text{global}} = W_r R_{\text{reliability}} + W_i R_{\text{inten}} + W_u R_{\text{util}} - g_{\text{Constant}}. \quad (5)$$

For the last action of the path, the two reward signals diverge in how they handle failure. The local reward R_{local} penalizes routes that fail to reach the intended destination by assigning a large negative value determined by $-l_{\text{Constant}}$, which is demand-specific and incorporates all five user-defined QoS weights W_c , W_t , W_r , W_i , and W_u (Table 4). The global reward R_{global} , by contrast, penalizes only actions that cause packet loss, using $-g_{\text{Constant}}$, which is derived solely from the network-condition weights W_r , W_i , and W_u . This asymmetry reflects the design intent: l_{Constant} encodes the full set of per-demand QoS preferences, while g_{Constant} captures only demand-independent network characteristics that generalize across different traffic demands.

E. Q-TABLE UPDATE

Both the local and global Q-tables are updated following the standard single-step on-policy SARSA update rule:

$$Q_{t+1}(s_t, a_t) = (1 - \alpha)Q_t(s_t, a_t) + \alpha (R + \gamma Q_t(s_{t+1}, a_{t+1})), \quad (6)$$

where α is the learning rate, γ is the discount factor, and R is the reward received after executing action a_t in state s_t (see Table 5 for parameter values). No eligibility traces or multi-step returns are employed. Updates are applied sequentially in the order actions were executed along the path, consistent with aggregate action selection since each state appears at most once per episode.

F. FINAL PATH EXTRACTION

Once all training episodes have been completed, the final routing path for a given traffic demand is extracted from the local Q-table using a greedy policy, i.e., by setting $\varepsilon = 0$ in FIND TEMP PATH. Starting from the source node, the algorithm repeatedly selects the outgoing action with the highest Q-value until the destination or a stopping condition (e.g., TTL) is reached. Under sufficient training, the resulting path is non-cyclic and reflects the highest estimated cumulative reward according to the user-defined QoS preferences.

VIII. EXPERIMENT SETUP AND RESULT ANALYSIS

We implemented two mechanisms for constructing network topologies used in our experiments. The first approach programmatically builds the topology using our provided methods, which sequentially generate the graph structure, instantiate nodes and links, and optionally assign initial link bandwidth utilization. The second approach allows users to define the topology using a JSON file following a specified schema; this file can be parsed using any standard open-source JSON parser.

RLSR-Routing is designed as a flexible framework for addressing the path-selection problem in SDN environments. It does not require prior knowledge of network parameters (e.g., link weights) nor depends on external machine learning libraries for training the reinforcement learning model. As long as the deployment environment can interact with the SDN controller and parse JSON-based topology descriptions, the framework can be readily integrated. Although this study primarily focuses on the design and implementation of the RL algorithm, we also developed supporting modules for network simulation. The implementation is written in Java; however, porting the framework to other programming languages would require minimal effort due to its modular design.

This section presents the comparative analysis and validation experiments, along with corresponding discussions.

A. SETTINGS FOR HYPERPARAMETERS

The authors of QAR in [16] conducted a series of experiments to examine the effects of the learning rate and the influence of long-term rewards on training

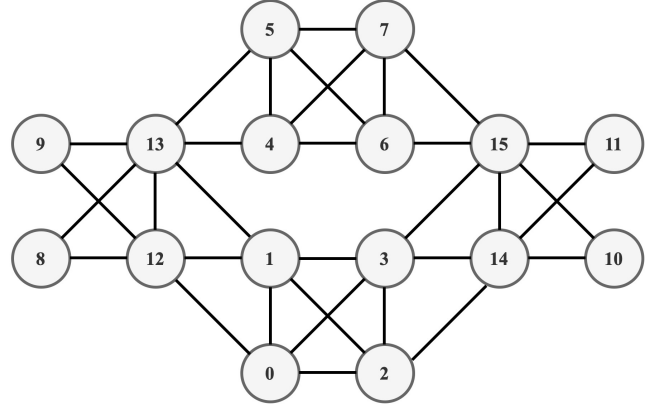


Fig. 3. 16 nodes topology (T7).

performance. Following their recommendations, we adopt the same hyperparameter configuration unless stated otherwise. The hyperparameters used in our experiments are summarized in Table 5.

B. COMPARATIVE STUDY WITH NON-RL ROUTING

To evaluate the effectiveness of RLSR-Routing, we conducted a comparative study against a non-RL-based routing algorithm, referred to as NR-Routing, which is currently deployed by a telecom solutions provider. Similar to RLSR-Routing, NR-Routing assigns paths for traffic flows between source–destination pairs; however, it relies on a non-learning-based agent that explores the traffic engineering space through greedy (largest-to-smallest) and/or random demand placement using multi-path segment-routing-based traffic engineering. Due to confidentiality constraints, further implementation details of NR-Routing cannot be disclosed.

The objective of this comparison is to assess the algorithms' *load-balancing performance*, with a specific focus on minimizing the maximum link utilization across a given network. The network topology used for this study, denoted as T7, is illustrated in Fig. 3. It contains 16 nodes, and each pair of adjacent nodes is connected by two unidirectional links. All links have a maximum bandwidth of 2.0 Mb/s, a reliability of 100%, and all nodes operate with a processing rate of 20 Mb/s.

1) NR-ROUTING WORKFLOW

NR-Routing is distributed as an executable program that provides two main functionalities:

- **Generation of traffic engineering instances**, including network topology files and sets of traffic demands.
- **Optimization of traffic placement** using segment-routing-based traffic steering.

We first used NR-Routing to randomly generate network instances and assign routing paths for each demand. The resulting paths were exported as a JSON file. We then curated these instances by removing unwanted demands and adjusting traffic volumes as needed; the modified demand sets served as NR-Routing's input for load-balancing

TABLE 6. Traffic distribution among nodes.

Source	Destination	Total Traffic	Number of Paths
4	14	400kb/s	2
1	5	400kb/s	2
0	6	1.0Mb/s	5
13	11	600kb/s	4
3	13	600kb/s	3
3	4	1.2Mb/s	6

evaluation. The selected traffic tunnels used in this study are summarized in Table 6.

2) RLSR-ROUTING INTERPRETATION OF NR-ROUTING TUNNELS

In RLSR-Routing, each NR-Routing tunnel is treated as n traffic demands, where n is the number of paths assigned to that tunnel by NR-Routing. Each individual demand receives an equal share of the tunnel's total traffic. For example, the six paths between nodes 3 and 4 correspond to six traffic demands of 200 kb/s each.

During training, the RL agent considers only link utilization when computing local rewards. Once a valid path is found for a demand, the associated traffic load is placed on the network along that path. This process repeats until all demands in the instance are routed and their traffic is applied to the network.

3) ROUTING PATHS GENERATED BY RLSR-ROUTING

Table 7 lists the routing paths generated by RLSR-Routing for each source–destination tunnel. The results demonstrate the algorithm's ability to construct valid and loop-free paths composed of consecutive links connecting the specified endpoints. All paths were verified to be feasible and free of routing loops, confirming that RLSR-Routing consistently identifies structurally sound routes for the given traffic demands.

4) LINK UTILIZATION ANALYSIS

Tables 8 and 9 report the *final per-link traffic loads* after all traffic demands have been routed and placed on the network by NR-Routing and RLSR-Routing, respectively. Each row corresponds to a directed link, and the reported value represents the aggregate bandwidth carried by that link, obtained by summing the traffic contributions of all routed demands traversing the link. We define the maximum link utilization as $U_{\max} = \max_{l \in E} (b_l / B_l)$, where b_l is the final sustained load on link l and B_l is its maximum bandwidth capacity.

For NR-Routing, link traffic values are exported in bits per second (b/s) in the JSON output and are converted to Mb/s for consistency with the rest of the paper. The largest raw value observed in Table 8 is 14.33333 Mb/s. This value corresponds to the aggregated traffic over multiple internal sub-path segments used by NR-Routing. Since each physical directed link in the evaluated topology has a maximum capacity of $B_l = 2.0$ Mb/s, the effective

TABLE 7. RLSR-routing generated paths for traffic demands.

Source-Destination	Path
4 to 14	i. 4 → 7 → 15 → 14
	ii. 4 → 6 → 15 → 14
1 to 5	i. 1 → 13 → 5
	ii. 1 → 13 → 5
0 to 6	i. 0 → 3 → 15 → 6
	ii. 0 → 2 → 14 → 15 → 6
	iii. 0 → 12 → 13 → 4 → 6
	iv. 0 → 3 → 15 → 7 → 6
	v. 0 → 1 → 3 → 15 → 6
13 to 11	i. 13 → 1 → 2 → 14 → 11
	ii. 13 → 1 → 3 → 14 → 11
	iii. 13 → 4 → 7 → 15 → 11
	iv. 13 → 5 → 6 → 15 → 11
3 to 13	i. 3 → 1 → 13
	ii. 3 → 0 → 12 → 13
	iii. 3 → 1 → 13
3 to 4	i. 3 → 15 → 7 → 4
	ii. 3 → 2 → 1 → 12 → 8 → 13 → 4
	iii. 3 → 14 → 15 → 7 → 4
	iv. 3 → 15 → 6 → 4
	v. 3 → 1 → 13 → 4
	vi. 3 → 0 → 12 → 13 → 4

TABLE 8. NR-routing's links with traffic (Bandwidth in Mb/S).

No.	Path	Bandwidth
1	R3(14) → R4(15)	8000000.0
2	R4(15) → R3(14)	4000000.0
3	B2(5) → B3(6)	316666.0
4	R1(12) → R2(13)	14333330.0
5	B3(6) → R4(15)	966666.0
6	R4(15) → B3(6)	500000.0
7	A2(1) → R2(13)	500000.0
8	A1(0) → R1(12)	933333.0
9	A4(3) → R4(15)	466666.0
10	R2(13) → B2(5)	866666.0
11	B1(4) → B3(6)	516666.0
12	B3(6) → B1(4)	300000.0
13	A4(3) → R3(14)	466666.0
14	R2(13) → B1(4)	1066666.0
15	A2(1) → R1(12)	500000.0
16	A3(2) → R3(14)	3333330.0
17	B1(4) → B4(7)	3500000.0
18	B4(7) → B1(4)	3000000.0
19	A4(3) → A1(0)	6000000.0
20	A1(0) → A3(2)	3333330.0
21	A4(3) → A2(1)	6000000.0
22	B4(7) → R4(15)	5000000.0
23	R4(15) → B4(7)	3000000.0
24	B2(5) → B4(7)	1500000.0
25	R4(15) → PE4(11)	6000000.0

per-link traffic used for utilization analysis is obtained by normalizing the aggregated value across these segments, yielding a maximum sustained load of $b_l = 1.433333$ Mb/s.

TABLE 9. RLSR-routing's links with traffic (Bandwidth in Mb/S).

No.	Path	Bandwidth
1	0→1	200000.0
2	0→2	200000.0
3	0→3	400000.0
4	0→12	600000.0
5	1→2	150000.0
6	1→3	350000.0
7	1→12	200000.0
8	1→13	1000000.0
9	2→1	200000.0
10	2→14	350000.0
11	3→0	400000.0
12	3→1	600000.0
13	3→2	200000.0
14	3→14	350000.0
15	3→15	1600000.0
16	4→6	400000.0
17	4→7	350000.0
18	5→6	150000.0
19	6→4	200000.0
20	6→15	350000.0
21	7→4	400000.0
22	7→6	200000.0
23	7→15	350000.0
24	8→13	200000.0
25	12→8	200000.0
26	12→13	600000.0
27	13→1	300000.0
28	13→4	950000.0
29	13→5	550000.0
30	14→11	300000.0
31	14→15	400000.0
32	15→6	800000.0
33	15→7	600000.0
34	15→11	300000.0
35	15→14	400000.0

Consequently, the maximum link utilization for NR-Routing is

$$U_{\max}(\text{NR}) = \frac{b_l}{B_l} = \frac{1.433333}{2.0} = 71.67\%.$$

In RLSR-Routing, each tunnel is decomposed into multiple equal-sized traffic demands, and each demand contributes a fixed bandwidth to every link along its selected path. The values in Table 9 are computed by summing these contributions per link. Although some links may temporarily carry up to 1.6 Mb/s during intermediate routing stages, the maximum *final sustained* per-link load after all demands are placed is $b_l = 1.0$ Mb/s. Thus, the maximum link utilization achieved by RLSR-Routing is

$$U_{\max}(\text{RLSR}) = \frac{b_l}{B_l} = \frac{1.0}{2.0} = 50.0\%.$$

Overall, Tables 8 and 9 serve to illustrate the distribution of traffic across individual links, while the reported utilization values are derived from the maximum final per-link loads. The results demonstrate that RLSR-Routing achieves a more balanced traffic distribution, reducing $U_{\max}$ from 71.67% to 50.0% compared with NR-Routing.

C. COST OF EXPLORATION DURING LEARNING PROCESS

Beyond load-balancing quality, aggregate action selection also reduces control-plane overhead relative to standard per-step SARSA. Under standard SARSA [16], each hop in an exploratory path requires one round-trip between the SDN controller and the network (one message to install the forwarding rule, one to collect QoS feedback), yielding $(2n)$ messages for an (n) -hop path. Under RLSR-Routing's aggregate action selection, the entire path is installed in a single batch and QoS data is collected in one pass, requiring $(n+1)$ messages per episode. For the paths generated in this study (Table 7), path lengths range from 2 to 6 hops, corresponding to reductions of 33%–58% in per-episode control-plane messages compared with standard SARSA.

We used a 30 nodes network as shown in Fig. 4 to test the efficacy of RLSR-Routing in the learning process. In topology T8, each pair of adjacent nodes are connected by two unidirectional links to support bi-directional communications. All nodes have a 100Mb/s processing rate, and all links have a 10Mb/s maximum bandwidth. Before any traffic demand's data been placed over the network, links used bandwidth are as follows:

- Link $l_{6,7}$, $l_{7,6}$, $l_{18,19}$, $l_{19,18}$: 0Mb/s used bandwidth;
- Link $l_{1,3}$, $l_{2,3}$: 5Mb/s used bandwidth;
- Link $l_{0,3}$, $l_{4,8}$, $l_{7,11}$, $l_{10,13}$, $l_{18,22}$: 9Mb/s used bandwidth;
- The rest of the links: 1Mb/s used bandwidth.

In this experiment, we gave RLSR-Routing nine traffic demands, each with 0.1Mb/s estimated traffic. The RL algorithm found one path for every given traffic demand, with traffic intensity and link utilization as considered QoS parameters ($W_i = W_u = 1$). A global Q-table was not used during this experiment since the experiment was not focused on convergence speed. After one path was returned from the RL algorithm, corresponding traffic demand data was placed over the network (i.e., all links on the path added 0.1Mb/s to their used bandwidth).

1) FINAL PATHS

Table 11 summarizes the final routing path returned by RLSR-Routing for every traffic demand. All routing paths are valid: every node in a path is only included once, and every path reaches the specified destination. During the learning process, RLSR-Routing gradually explored the network, so that highly utilized links (like link $l_{0,3}$, $l_{4,8}$, $l_{7,11}$, $l_{10,13}$, $l_{18,22}$) and nodes with relatively high traffic intensity (like node 3, 8, 11, 13, 22) are excluded in final paths. As a result, all the paths include the same set of nodes, node 4, 7, 6, 10, 14, 18, 19, 23 as intermediate nodes between source and destination. Although these are not the shortest paths

TABLE 10. Episode number when RL converged.

Demand	Control	$\gamma = 0.3$	$\gamma = 0.5$	$\gamma = 0.7$	$\gamma = 0.9$
0 → 26	25	25	25	25	25
1 → 26	27	26	25	21	18
2 → 26	30	28	38	17	17
0 → 27	30	21	24	23	20
1 → 27	27	24	25	21	20
2 → 27	31	25	26	24	25
0 → 28	29	22	25	27	21
1 → 28	30	26	28	29	22
2 → 28	34	34	32	32	24

TABLE 11. Final path for traffic demands.

Demand	Final Path
0 → 26	0 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 26
1 → 26	1 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 26
2 → 26	2 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 26
0 → 27	0 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 27
1 → 27	1 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 27
2 → 27	2 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 27
0 → 28	0 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 28
1 → 28	1 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 28
2 → 28	2 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 28

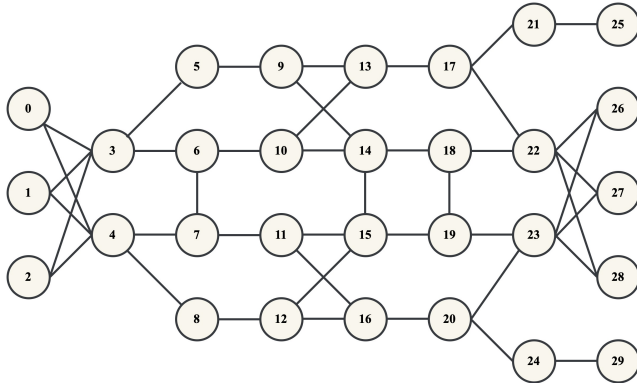


Fig. 4. 30 nodes topology (T8).

in terms of hop-count, they are the user preferred paths in terms of link utilization and traffic intensity.

2) PATH LENGTH VS EPISODE NUMBER

Fig. 5 demonstrated when RLSR-Routing was finding the path for each traffic demand, the length (i.e., number of hops) of temporary paths (denote as TempPath) generated in the first 50 episodes. None of the temporary paths would result in a packet inside a loop, and most of the paths have less than 20 nodes included. Although at the beginning of the learning process, especially during the first 20 episodes, some temporary paths failed to reach the desired destination, the agent quickly learned to avoid choosing actions that lead to a dead end. For most of the testing traffic demands, RLSR-Routing was able to converge around 30 episodes;

TABLE 12. Final paths for all test groups (Identical for $\gamma = 0.3, 0.5, 0.7, 0.9$).

Demand	Final Path (Common Across All Test Groups)
0 → 26	0 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 26
1 → 26	1 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 26
2 → 26	2 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 26
0 → 27	0 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 27
1 → 27	1 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 27
2 → 27	2 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 27
0 → 28	0 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 28
1 → 28	1 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 28
2 → 28	2 → 4 → 7 → 6 → 10 → 14 → 18 → 19 → 23 → 28

after that, the temporary paths produced in each episode are the same as the final paths.

3) GLOBAL Q-TABLE AND EFFECTS OF γ VALUE

We evaluate whether initializing local Q-tables using a global Q-table can accelerate the convergence of RLSR-Routing. The experiments were conducted on the same network topology (T8) and with the same initial link utilizations described in Section VIII. The same set of traffic demands and QoS weight configurations were also used. The only difference in this study is that the global Q-table was reused to initialize each local Q-table before learning began.

To examine how the discount factor γ , which determines the importance of long-term rewards, influences the effectiveness of the global Q-table, we configured five experimental groups. The experiment described in Section VIII-C did not use a global Q-table, whereas Test Groups I-IV updated the global Q-table using γ values of 0.3, 0.5, 0.7, and 0.9, respectively. For each group, we recorded the final routing paths for every traffic demand as well as the number of episodes required for convergence.

The final routing paths produced by all test groups are summarized in Table 12. A key observation from this table is that all four test groups converged to *identical* routing paths for every traffic demand, regardless of the value of γ . This finding demonstrates that incorporating a global Q-table does not alter the optimal routing policy learned by RLSR-Routing; rather, it primarily affects the rate at which the algorithm converges.

In other words, although different γ values influence how strongly future QoS rewards shape Q-value updates, they consistently guide the algorithm toward the same QoS-preferred, loop-free routing paths. This shows that the optimal policy is robust across varying levels of long-term reward weighting. Thus, the principal benefit of the global Q-table lies not in modifying the final routing solution, but in enabling faster learning across multiple routing tasks by transferring previously accumulated QoS knowledge.

4) RL CONVERGENCE SPEED

Table 10 reports, for each traffic demand, the number of training episodes required for convergence under different

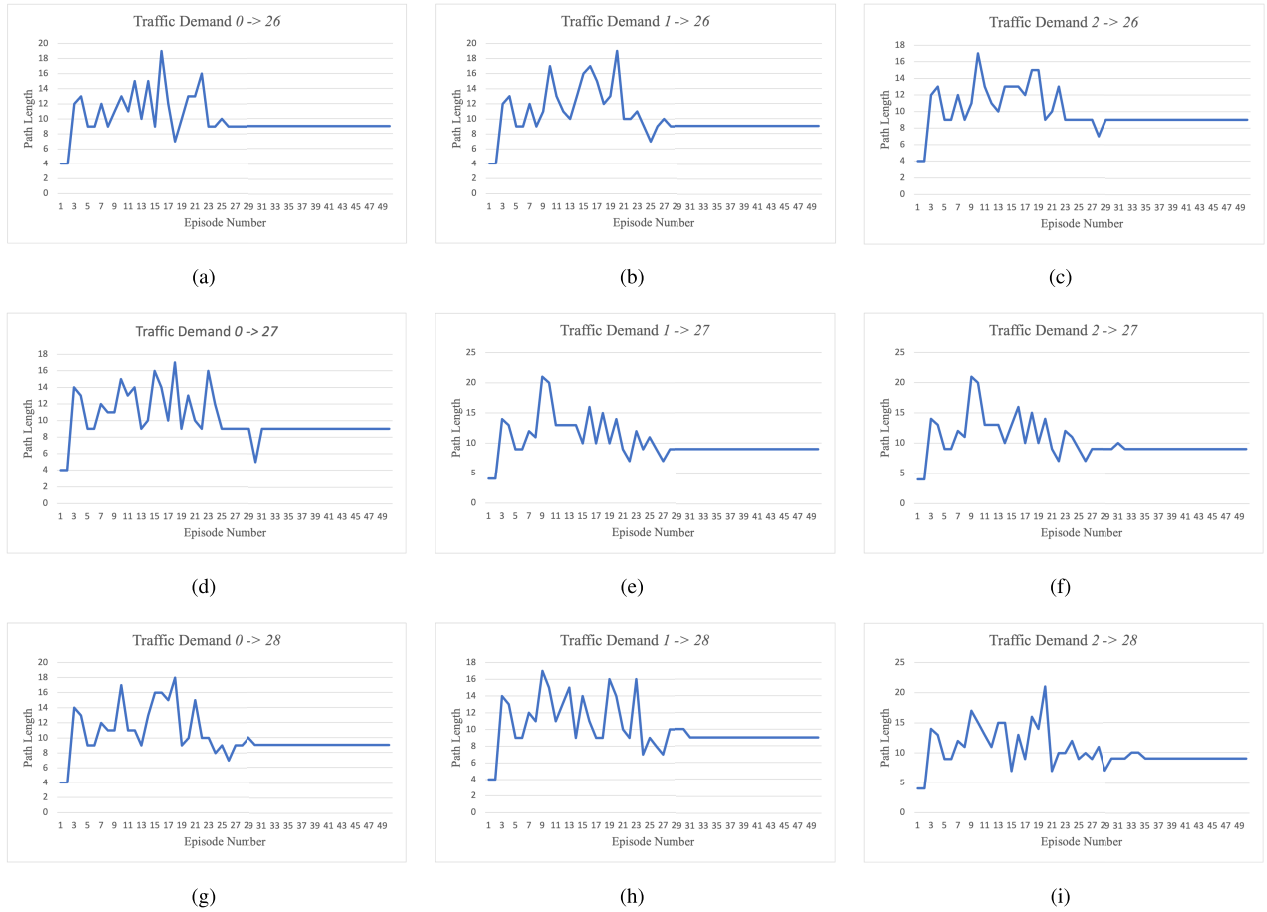


Fig. 5. Evolution of temporary path (TempPath) lengths during the first 50 learning episodes in RLSR-Routing. Subfigures (a)–(c) correspond to demands $0 \rightarrow 26$, $1 \rightarrow 26$, and $2 \rightarrow 26$; (d)–(f) to demands $0 \rightarrow 27$, $1 \rightarrow 27$, and $2 \rightarrow 27$; and (g)–(i) to demands $0 \rightarrow 28$, $1 \rightarrow 28$, and $2 \rightarrow 28$. Despite occasional failures to reach the destination in early episodes, the agent rapidly learns loop-free paths and converges to stable routing solutions by approximately the 30th episode.

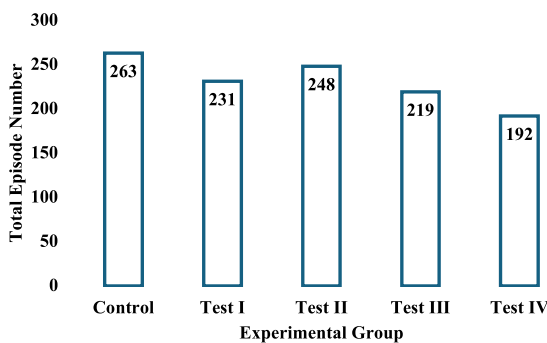


Fig. 6. Total episode numbers when RL converged for all demands.

experimental configurations. According to the experimentation, the test groups generally required fewer episodes to converge for the same traffic demands, indicating that the use of a global Q-table accelerates learning.

Figure 6 summarizes the total number of episodes required for convergence across all traffic demands under each experimental setting. This value is not an average;

rather, it is computed as the cumulative number of episodes required for the algorithm to successfully converge for all demands:

$$E_{\text{total}} = \sum_{i=1}^N E_i,$$

where E_i denotes the number of episodes needed for convergence of the i -th traffic demand, and N is the total number of demands. Since the same training configuration and convergence criteria are applied deterministically to all demands within each experimental group, the reported total episode numbers are fixed for a given setting.

IX. CONCLUSION

In this research, we developed RLSR-Routing, a reinforcement learning-based routing algorithm designed for SR-enabled SDN environments. The framework adapts the on-policy SARSA algorithm by introducing an aggregated action-selection mechanism and a dual-reward scheme. To address potential packet loss or failures to reach the intended destination during training, we incorporated additional update steps to adjust the Q-table for unsuccessful actions.

Experimental results demonstrate that RLSR-Routing is capable of identifying user-preferred paths for given traffic demands based on customizable QoS requirements.

However, several limitations remain. First, the greedy action-selection strategy may cause the algorithm to converge to suboptimal paths in certain scenarios, an effect that may be amplified by the moderate network sizes and limited traffic demands considered here, which could restrict the diversity of observed routing behaviors. Second, RLSR-Routing currently computes paths sequentially without considering interaction effects on remaining unassigned demands; in practical deployments, it should be applied to the full set of user traffic demands. Third, while the 16- and 30-node topologies used are sufficient to demonstrate the core algorithmic properties of RLSR-Routing, they do not capture the complexity of larger real-world networks or public benchmark topologies such as GEANT and NSFNET; evaluation under time-varying and burst traffic conditions similarly remains an important direction for future work. Fourth, although RLSR-Routing's tabular Q-table structure supports incremental adaptation to topology changes — for a failure on link $l_{i,j}$, the entries $Q[i][j]$ in both local and global Q-tables can be set to the minimum value without requiring full retraining — a full empirical evaluation under dynamic conditions including link failures, capacity fluctuations, and evolving demand matrices on larger and more heterogeneous networks remains outstanding. The most closely related SDN-based baseline, Hossain and Wei [45], also does not address these scalability and robustness dimensions, suggesting they represent open challenges for the broader research community.

As discussed earlier, several enhancements can further improve RLSR-Routing's performance and scalability. These include designing a more robust action-selection policy and allowing greater flexibility in initializing local Q-tables from a global Q-table. For future work, we plan to evaluate RLSR-Routing in more realistic traffic-engineering scenarios, such as networks with up to 1,000 nodes and thousands of traffic demands. Additionally, future experiments should assess the algorithm's ability to discover user-preferred paths across a broader range of QoS metrics (e.g., link reliability), rather than relying solely on link utilization and traffic intensity. Finally, we intend to extend the RL component to support parallel path computation for traffic-demand matrices, enabling more efficient and scalable operation.

REFERENCES

- [1] CISCO Systems. (2020). *CISCO Annual Internet Report (2018–2023)*. Accessed: Apr. 1, 2023. [Online]. Available: <http://surl.li/binfj>
- [2] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*. Boston, MA, USA: Pearson, 2017.
- [3] J. Moy, *OSPF Version 2*, document RFC 2328, 1998.
- [4] C. Hedrick, *Routing Information Protocol*, document RFC 1058, 1988.
- [5] Y. Guo, Z. Wang, X. Yin, X. Shi, and J. Wu, "Traffic engineering in SDN/OSPF hybrid network," in *Proc. IEEE 22nd Int. Conf. Netw. Protocols*, Oct. 2014, pp. 563–568.
- [6] Y. Wang, Z. Wang, and L. Zhang, "Internet traffic engineering without full mesh overlaying," in *Proc. Conf. Comput. Commun. 20th Annu. Joint Conf. IEEE Comput. Commun. Soc. (IEEE INFOCOM)*, vol. 1, Apr. 2001, pp. 565–571.
- [7] M. Karakus and A. Dursesi, "Quality of service (QoS) in software defined networking (SDN): A survey," *J. Netw. Comput. Appl.*, vol. 80, pp. 200–218, Feb. 2017.
- [8] R. Masoudi and A. Ghaffari, "Software defined networks: A survey," *J. Netw. Comput. Appl.*, vol. 67, pp. 1–25, May 2016.
- [9] I. T. Haque and N. Abu-Ghazaleh, "Wireless software defined networking: A survey and taxonomy," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 4, pp. 2713–2737, 2016.
- [10] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, *Segment Routing Architecture*, document RFC 8402, 2018.
- [11] C. Filsfils, N. K. Nainar, C. Pignataro, J. C. Cardona, and P. Francois, "The segment routing architecture," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2015, pp. 1–6.
- [12] S. Previdi, C. Filsfils, B. Decraene, S. Litkowski, M. Horneffer, and R. Shakir, *Source Packet Routing in Networking (SPRING) Problem Statement and Requirements*, document RFC 7855, 2016.
- [13] R. Boutaba et al., "A comprehensive survey on machine learning for networking: Evolution, applications and research opportunities," *J. Internet Services Appl.*, vol. 9, no. 1, pp. 1–99, Dec. 2018.
- [14] C. Wu, S. Ohzahata, and T. Kato, "Flexible, portable, and practicable solution for routing in VANETs: A fuzzy constraint Q-learning approach," *IEEE Trans. Veh. Technol.*, vol. 62, no. 9, pp. 4251–4263, Nov. 2013.
- [15] C. Wu, T. Yoshinaga, Y. Ji, and Y. Zhang, "Computational intelligence inspired data delivery for vehicle-to-roadside communications," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12038–12048, Dec. 2018.
- [16] S.-C. Lin, I. F. Akyildiz, P. Wang, and M. Luo, "QoS-aware adaptive routing in multi-layer hierarchical software defined networks: A reinforcement learning approach," in *Proc. IEEE Int. Conf. Services Comput. (SCC)*, Jun. 2016, pp. 25–33.
- [17] R. E. Ali, B. Erman, E. Bastug, and B. Cilli, "Hierarchical deep double Q-routing," in *Proc. ICC - IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–7.
- [18] Z. Mammeri, "Reinforcement learning based routing in networks: Review and classification of approaches," *IEEE Access*, vol. 7, pp. 55916–55950, 2019.
- [19] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [20] T. Hu and Y. Fei, "QELAR: A machine-learning-based adaptive routing protocol for energy-efficient and lifetime-extended underwater sensor networks," *IEEE Trans. Mobile Comput.*, vol. 9, no. 6, pp. 796–809, Jun. 2010.
- [21] S. Z. Jafarzadeh and M. H. Y. Moghaddam, "Design of energy-aware QoS routing algorithm in wireless sensor networks using reinforcement learning," in *Proc. 4th Int. Conf. Comput. Knowl. Eng. (ICCKE)*, Oct. 2014, pp. 722–727.
- [22] B. Debowski, P. Spachos, and S. Areibi, "Q-learning enhanced gradient based routing for balancing energy consumption in WSNs," in *Proc. IEEE 21st Int. Workshop Comput. Aided Model. Design Commun. Links Netw. (CAMAD)*, Oct. 2016, pp. 18–23.
- [23] G. Santhi, A. Nachiappan, M. Z. Ibrahim, R. Raghunadhane, and M. K. Favas, "Q-learning based adaptive QoS routing protocol for MANETs," in *Proc. Int. Conf. Recent Trends Inf. Technol. (ICRITIT)*, Jun. 2011, pp. 1233–1238.
- [24] C. Perkins, E. Belding-Royer, and S. Das, *Ad Hoc On-Demand Distance Vector (AODV) Routing*, document RFC 3561, 2003.
- [25] W.-S. Jung, J. Yim, and Y.-B. Ko, "QGeo: Q-learning-based geographic ad hoc routing protocol for unmanned robotic networks," *IEEE Commun. Lett.*, vol. 21, no. 10, pp. 2258–2261, Oct. 2017.
- [26] T. Hendriks, M. Camelo, and S. Latré, "Q2-routing: A qos-aware Q-routing algorithm for wireless ad hoc networks," in *Proc. 14th Int. Conf. Wireless Mobile Comput., Netw. Commun. (WiMob)*, Oct. 2018, pp. 108–115.
- [27] F. Li, X. Song, H. Chen, X. Li, and Y. Wang, "Hierarchical routing for vehicular ad hoc networks via reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1852–1865, Feb. 2019.
- [28] M. Nurchis, R. Bruno, M. Conti, and L. Lenzini, "A self-adaptive routing paradigm for wireless mesh networks based on reinforcement learning," in *Proc. 14th ACM Int. Conf. Model., Anal. Simul. wireless mobile Syst.*, Oct. 2011, pp. 197–204.

- [29] M. Boushaba, A. Hafid, A. Belbekkouche, and M. Gendreau, "Reinforcement learning based routing in wireless mesh networks," *Wireless Netw.*, vol. 19, no. 8, pp. 2079–2091, Nov. 2013.
- [30] K. Tang, C. Li, H. Xiong, J. Zou, and P. Frossard, "Reinforcement learning-based opportunistic routing for live video streaming over multi-hop wireless networks," in *Proc. IEEE 19th Int. Workshop Multimedia Signal Process. (MMSP)*, Oct. 2017, pp. 1–6.
- [31] D. M. Casas-Velasco, O. M. C. Rendon, and N. L. S. da Fonseca, "Intelligent routing based on reinforcement learning for software-defined networking," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 1, pp. 870–881, Mar. 2021.
- [32] Z. Yuan, P. Zhou, S. Wang, and X. Zhang, "Research on routing optimization of SDN network using reinforcement learning method," in *Proc. 2nd Int. Conf. Saf. Produce Informatization (IICSPI)*, Nov. 2019, pp. 442–445.
- [33] S. Yang, H. Shi, and H. Zhang, "Dynamic load balancing of multiple controller based on intelligent collaboration in SDN," in *Proc. Int. Conf. Comput. Vis., Image Deep Learn. (CVIDL)*, Jul. 2020, pp. 354–359.
- [34] S. Kim, J. Son, A. Talukder, and C. S. Hong, "Congestion prevention mechanism based on Q-learning for efficient routing in SDN," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, 2016, pp. 124–128.
- [35] D. Tennakoon, S. Karunaratna, and B. Udugama, "Q-learning approach for load-balancing in software defined networks," in *Proc. Moratuwa Eng. Res. Conf. (MERCon)*, May 2018, pp. 1–6.
- [36] J. Rischke, P. Sossalla, H. Salah, F. H. P. Fitzek, and M. Reisslein, "QR-SDN: Towards reinforcement learning states, actions, and rewards for direct flow routing in software-defined networks," *IEEE Access*, vol. 8, pp. 174773–174791, 2020.
- [37] S. Sendra, A. Rego, J. Lloret, J. M. Jimenez, and O. Romero, "Including artificial intelligence in a routing protocol using software defined networks," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, May 2017, pp. 670–674.
- [38] C. Wang, L. Zhang, Z. Li, and C. Jiang, "SDCoR: Software defined cognitive routing for Internet of Vehicles," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3513–3520, Oct. 2018.
- [39] J. A. Boyan and M. L. Littman, "Packet routing in dynamically changing networks: A reinforcement learning approach," in *Proc. Adv. Neural Inf. Process. Syst.*, 1993, p. 6.
- [40] C. Yu, J. Lan, Z. Guo, and Y. Hu, "DROM: Optimizing the routing in software-defined networks with deep reinforcement learning," *IEEE Access*, vol. 6, pp. 64533–64539, 2018.
- [41] Y. Tian et al., "Traffic engineering in partially deployed segment routing over IPv6 network with deep reinforcement learning," *IEEE/ACM Trans. Netw.*, vol. 28, no. 4, pp. 1573–1586, Aug. 2020.
- [42] P. Almasan, S. Xiao, X. Cheng, X. Shi, P. Barlet-Ros, and A. Cabellos-Aparicio, "ENERO: Efficient real-time WAN routing optimization with deep reinforcement learning," 2021, *arXiv:2109.10883*.
- [43] Q. Fu, E. Sun, K. Meng, M. Li, and Y. Zhang, "Deep Q-learning for routing schemes in SDN-based data center networks," *IEEE Access*, vol. 8, pp. 103491–103499, 2020.
- [44] H. Hasselt, "Double Q-learning," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 23, 2010.
- [45] M. B. Hossain and J. Wei, "Reinforcement learning-driven QoS-aware intelligent routing for software-defined networks," in *Proc. IEEE Global Conf. Signal Inf. Process. (GlobalSIP)*, Nov. 2019, pp. 1–5.



SAJAL SAHA (Senior Member, IEEE) received the Graduate (Hons.) and M.Sc. degrees from Brock University and the Ph.D. degree in computer science from Western University, Canada. He is currently an Assistant Professor of computer science with the University of Northern British Columbia, Canada, and the Founder of the INFORM Laboratory. He supervises graduate students through the INFORM Laboratory. His work is supported by the NSERC Discovery Grant and MITACS programs. He has published in leading IEEE journals and conferences. His research interests include internet traffic analysis, cyber-attack detection and mitigation, privacy-preserving federated learning, and AI-driven network security. He serves on the technical program committees of several international conferences and journals in cybersecurity and networking. He was nominated for the Governor General's Gold Medal at Western University.



ANWAR HAQUE received the master's degree in computer science and the Ph.D. degree in electrical and computer engineering from the University of Waterloo. He is currently an Associate Professor and the Undergraduate Chair of the Department of Computer Science, The University of Western Ontario (Western University), Canada. Before joining Western University, he was an Associate Director at Bell Canada. At Western University, he served as the Industry Expert-in-Residence at the Faculty of Science, a member of Western's Senate, and on the Inaugural Advisory Committee for the newly established Bell-Western 5G Research Centre. Additionally, he is the Founder and the CEO of Bamboo Innovations Inc., a successful tech venture in the smart systems industry. His collaborative research grants are valued at more than 15 million. He has authored/co-authored more than 100 peer-reviewed research publications in leading journals and conferences, holds several patents/licenses, and supervised more than 100 HQPs. His research interests include next-gen communication networks, the IoT, cyber-security, and applied AI, focusing on developing secure and resilient autonomous cyber-physical systems (CPS).



WANG WUMIAN received the bachelor's degree in microbiology and immunology from The University of British Columbia in 2019 and the master's degree in computer science from The University of Western Ontario in 2022. Alongside his studies, he worked as a Teaching Assistant in a computer science course in 2022. He won the Research Scholarship for the year 2021–2022.



GREG SIDEBOTTOM received the Ph.D. degree in computer science from Simon Fraser University. He is currently an Applied Scientist with Amazon Web Services. He is also the Architect of pioneering software-defined networks and network function virtualization products.